

Unidad 2: Distintos modelos NoSQL

Clase 7 – Datos distribuidos y eventos masivos: modelo tabular y CQL

Objetivo

Diseñar estructuras de datos orientadas a consultas en entornos distribuidos, utilizando CQL para comprender el manejo de eventos masivos, series temporales y los compromisos entre escalabilidad y consistencia.

Tema

- Modelo tabular / wide-column.
- CQL – Cassandra Query Language
 - Diseño orientado a consultas
 - Claves de partición
 - Time series y eventos
 - Consistencia y escalabilidad
- Ejemplos aplicados a logs y eventos masivos.

Datos distribuidos y eventos masivos: modelo tabular y CQL

Bases de datos de columna ancha

Las bases de datos de columna ancha, también llamadas bases de datos tabulares, columnares extendidas o wide-column databases, son un tipo de base de datos NoSQL diseñado para almacenar grandes volúmenes de información distribuida. Aunque utilizan conceptos como tablas, filas y columnas, no deben confundirse con una base de datos relacional tradicional.

En una base relacional, una tabla suele tener una estructura fija: todas las filas comparten las mismas columnas, existen relaciones entre tablas, se usan claves primarias y foráneas, y las consultas pueden combinar información mediante JOIN. En cambio, en una base de columna ancha, la estructura es más flexible y está orientada a distribuir datos eficientemente entre varios nodos. El material base describe este modelo como una estructura tabular en la que cada fila puede tener una configuración diferente de columnas, con capacidad para manejar muchas columnas y grandes volúmenes de datos .

Este tipo de modelo aparece como respuesta a problemas donde una base relacional puede volverse costosa o poco eficiente:

- Registro masivo de eventos.
- Logs de aplicaciones.
- Historial de actividad de usuarios.
- Series temporales.

- Sensores IoT.
- Trazabilidad de transacciones.
- Monitoreo de sistemas.
- Eventos de seguridad.
- Clics, búsquedas, sesiones y comportamiento digital.
- Datos que deben escribirse muy rápido y consultarse por patrones conocidos.

Por ejemplo, una plataforma digital puede generar millones de eventos por día:

```
usuario_1001 inició sesión a las 08:10
usuario_1001 realizó una búsqueda a las 08:12
usuario_1001 agregó un producto al carrito a las 08:15
usuario_1001 realizó un pago rechazado a las 08:18
usuario_1001 intentó ingresar desde otro dispositivo a las 08:23
```

Guardar estos datos en una tabla relacional tradicional no es imposible. Sin embargo, si el volumen crece demasiado y se necesita consultar rápidamente por usuario, fecha, tipo de evento o dispositivo, el diseño relacional puede requerir índices complejos, particionamiento, joins, optimizaciones y mantenimiento permanente.

Las bases de columna ancha buscan resolver este problema desde otra lógica: distribuir los datos desde el diseño y organizarlos según las consultas que se necesitan responder.

De la tabla relacional a la tabla orientada a consulta

En SQL Server, el punto de partida habitual es el modelo de datos. Se identifican entidades, atributos y relaciones. Luego se normaliza el modelo para evitar redundancia. Por ejemplo, en un sistema de eventos se podrían crear tablas como:

Usuarios
Eventos
TiposEvento
Dispositivos
Sesiones

Después, mediante consultas SQL, se combinan esas tablas para obtener la información necesaria.

En Cassandra, el razonamiento cambia. El punto de partida no es solamente “qué entidades existen”, sino principalmente: **¿Qué consulta necesito responder rápido?**

Esto modifica el diseño. Una tabla en Cassandra no siempre representa una entidad del negocio. Muchas veces representa una consulta específica.

Por ejemplo, si se necesita consultar los eventos de un usuario por día, podría existir una tabla llamada:

eventos_por_usuario_dia

Si también se necesita consultar eventos por tipo y fecha, podría existir otra tabla:

eventos_por_tipo_fecha

Y si se necesita consultar eventos sospechosos por nivel de riesgo, podría existir otra:

eventos_sospechosos_por_fecha

El mismo evento puede repetirse en varias tablas. Esto, que en un modelo relacional parecería una mala práctica por redundancia, en Cassandra puede ser una decisión correcta de diseño. La desnormalización permite evitar joins y lecturas costosas. El material de modelado de Cassandra remarca precisamente que no se comienza con el modelo de datos, sino con el modelo de consulta, y que las tablas se crean para soportar las rutas de consulta más frecuentes .

La idea central es: *En Cassandra no se diseña una tabla para guardar “todo”. Se diseña una tabla para responder bien una pregunta.*

Cassandra: una base distribuida para grandes volúmenes

Apache Cassandra es una base de datos NoSQL de columna ancha, distribuida y escalable horizontalmente. Está pensada para escenarios donde se necesita alta disponibilidad, tolerancia a fallos y manejo de grandes volúmenes de escritura y lectura. – *CQL: Lenguaje de consulta de Casandra*: es el lenguaje oficial de Apache Cassandra para interactuar con la base de datos. Su sintaxis es muy similar a SQL, permitiendo crear esquemas (DDL) y gestionar datos (DML) de forma sencilla, pero está optimizado para la arquitectura distribuida y NoSQL de Cassandra

Cassandra se utiliza especialmente cuando los datos deben distribuirse entre varios nodos. Un nodo es una máquina del clúster que almacena parte de los datos. El clúster completo está formado por varios nodos que trabajan de manera coordinada. A diferencia de arquitecturas con un servidor central principal, Cassandra trabaja con una arquitectura distribuida donde los nodos se comunican entre sí y cualquiera de ellos puede recibir una operación. El material de referencia indica que los clientes pueden conectarse a cualquier nodo y que ese nodo actúa como coordinador de la operación frente al resto de los nodos involucrados .

Esto permite comprender una de sus principales fortalezas: la escalabilidad horizontal. Si el volumen de datos o consultas crece, la solución no depende únicamente de hacer más potente un servidor, sino de agregar más nodos al clúster.

Cassandra resulta especialmente adecuada cuando se combinan estas condiciones:

- Mucho volumen de datos.
- Muchas escrituras.
- Datos distribuidos.

- Consultas conocidas de antemano.
- Necesidad de alta disponibilidad.
- Tolerancia a fallos.
- Lecturas rápidas por claves bien diseñadas.

No es la opción ideal cuando se necesitan muchas consultas libres, joins complejos, integridad referencial estricta o transacciones ACID tradicionales. El propio material de bases tabulares advierte que no es conveniente para sistemas que requieren transacciones ACID para lecturas y escrituras, ni para consultas agregadas tradicionales que deberían resolverse como en un motor relacional .

El cambio de mentalidad: de normalizar a consultar

El cambio más importante al trabajar con Cassandra no es la sintaxis, sino la forma de pensar el diseño.

En una base relacional, el razonamiento habitual es:

Entidades → Relaciones → Normalización → Consultas

En Cassandra, el razonamiento recomendado es:

Consultas → Clave de partición → Clave de ordenamiento → Tabla optimizada

Esto significa que antes de crear tablas hay que definir las preguntas que el sistema debe responder.

Por ejemplo:

- ¿Cuáles fueron los últimos eventos de un usuario en un día?
- ¿Cuáles fueron los eventos sospechosos de una fecha?
- ¿Cuántos intentos fallidos tuvo un usuario en una ventana de tiempo?
- ¿Qué transacciones se registraron para un cliente?
- ¿Qué eventos ocurrieron en determinado dispositivo?
- ¿Qué errores generó una aplicación durante la última hora?

Cada una de estas preguntas puede requerir una tabla distinta.

Un error común sería intentar crear una única tabla general:

eventos

Con columnas como:

evento_id, usuario_id, fecha, tipo_evento, ip, dispositivo, riesgo, detalle

Y luego pretender consultar libremente por cualquier columna, como se haría en SQL. En Cassandra esto no funciona bien, porque las consultas deben estar alineadas con la clave primaria de la tabla. Por eso, el diseño debe anticipar el acceso a los datos.

La frase clave es: *Primero se diseña la consulta; después se diseña la tabla.*

Clave primaria, clave de partición y clave de clustering

La clave primaria en Cassandra cumple un rol fundamental. No solo identifica registros. También define cómo se distribuyen y ordenan los datos.

Una clave primaria puede tener dos componentes:

Clave de partición: Determina en qué partición se almacenan los datos. En términos simples, indica cómo se agrupan físicamente los datos y cómo se distribuyen entre los nodos.

Clave de clustering: Determina cómo se ordenan los datos dentro de una misma partición.

Row key es el equivalente a la clave primaria, compuesta por clave de particionado y clave de agrupamiento: la primera permite almacenar juntas las filas con el mismo valor de partición, mientras la segunda determina el orden físico de almacenamiento dentro de esa partición .

Ejemplo conceptual:

```
CREATE TABLE eventos_por_usuario_dia (
  usuario_id text,
  fecha date,
  evento_ts timestamp,
  evento_id uuid,
  tipo_evento text,
  ip text,
  dispositivo text,
  resultado text,
  PRIMARY KEY ((usuario_id, fecha), evento_ts, evento_id)
) WITH CLUSTERING ORDER BY (evento_ts DESC);
```

En esta tabla:

(usuario_id, fecha) = clave de partición
evento_ts, evento_id = claves de clustering

La clave de partición agrupa todos los eventos de un usuario en una fecha determinada. La clave de clustering ordena esos eventos por momento de ocurrencia. El evento_id ayuda a garantizar unicidad cuando dos eventos pudieran tener el mismo timestamp.

Esta tabla está pensada para una consulta como:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03'
LIMIT 20;
```

El diseño permite recuperar rápidamente los eventos de un usuario en un día, ordenados desde el más reciente al más antiguo.

Pero esta tabla no está diseñada para consultar directamente:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE tipo_evento = 'LOGIN_FALLIDO';
```

La columna tipo_evento no forma parte de la clave de partición. Por lo tanto, Cassandra no puede usarla como ruta principal de acceso eficiente. Para esa necesidad convendría otra tabla.

Series temporales y eventos masivos

Uno de los usos más frecuentes de Cassandra es el almacenamiento de eventos en el tiempo. Una serie temporal es una secuencia de datos registrados en distintos momentos. Cada registro tiene una marca de tiempo y representa algo que ocurrió.

Ejemplos:

- Lecturas de temperatura de un sensor.
- Intentos de login de un usuario.
- Transacciones de una billetera virtual.
- Eventos de navegación en una aplicación.
- Cambios de estado de un pedido.
- Alertas de seguridad.
- Errores generados por un servidor.

En estos casos, el tiempo no es un dato más: es parte central del diseño.

Una mala decisión sería crear una partición demasiado grande. Por ejemplo:

```
PRIMARY KEY (usuario_id, evento_ts)
```

Este diseño agruparía todos los eventos históricos de un usuario en una única partición. Si el usuario genera millones de eventos, esa partición crecería demasiado.

Una mejor alternativa suele ser dividir por períodos:

PRIMARY KEY ((usuario_id, fecha), evento_ts)

De esta forma, los eventos se agrupan por usuario y día. También podría usarse usuario y mes, dispositivo y día, sensor y hora, comercio y fecha, según el volumen esperado.

La decisión depende de la cantidad de datos. No es lo mismo registrar 10 eventos por usuario por día, que registrar: 500.000 eventos por dispositivo por hora

En Cassandra, el diseño debe evitar dos extremos:

- Particiones demasiado grandes, porque concentran muchos datos en un mismo grupo.
- Particiones demasiado pequeñas o dispersas, porque obligan a buscar en demasiados lugares.

El objetivo es lograr particiones razonables: suficientemente grandes para agrupar datos relacionados, pero no tan grandes como para afectar el rendimiento.

Primer acercamiento a CQL

CQL, Cassandra Query Language, es el lenguaje utilizado para crear y consultar estructuras en Cassandra. Su sintaxis se parece a SQL, pero su lógica no es la misma.

CQL permite crear keyspaces, tablas, insertar datos, consultar, actualizar y eliminar registros. La referencia de CQL muestra comandos como CREATE KEYSPACE, CREATE TABLE, INSERT, UPDATE, DELETE, SELECT, ORDER BY, LIMIT y ALLOW FILTERING .

Un CREATE TABLE básico podría ser:

```
CREATE TABLE eventos_por_usuario_dia (
  usuario_id text,
  fecha date,
  evento_ts timestamp,
  evento_id uuid,
  tipo_evento text,
  canal text,
  ip text,
  dispositivo text,
  resultado text,
  riesgo int,
  detalle text,
  PRIMARY KEY ((usuario_id, fecha), evento_ts, evento_id)
) WITH CLUSTERING ORDER BY (evento_ts DESC);
```

Este código no solo crea una tabla. También define la forma en que Cassandra organizará los datos.

La parte más importante es: **PRIMARY KEY ((usuario_id, fecha), evento_ts, evento_id)**

La doble estructura indica:

- Clave de partición: usuario_id + fecha
- Claves de clustering: evento_ts + evento_id

El orden descendente permite recuperar primero los eventos más recientes:

WITH CLUSTERING ORDER BY (evento_ts DESC);

Una consulta válida sería:

```
SELECT tipo_evento, evento_ts, ip, dispositivo, resultado
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03'
LIMIT 10;
```

Esta consulta es válida porque usa la clave de partición completa.

Una consulta problemática sería:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE riesgo >= 80;
```

Aunque riesgo exista como columna, no fue elegida como clave de partición ni clustering. Por lo tanto, Cassandra no puede usarla como acceso principal eficiente.

Para consultar eventos por riesgo, se debería pensar otra tabla, por ejemplo:

```
CREATE TABLE eventos_por_riesgo_fecha (
    nivel_riesgo int,
    fecha date,
    evento_ts timestamp,
    evento_id uuid,
    usuario_id text,
    tipo_evento text,
    ip text,
    detalle text,
    PRIMARY KEY ((nivel_riesgo, fecha), evento_ts, evento_id)
) WITH CLUSTERING ORDER BY (evento_ts DESC);
```

La misma información puede estar duplicada, pero ahora responde otra pregunta:

```
SELECT *
FROM eventos_por_riesgo_fecha
```



```
WHERE nivel_riesgo = 90  
AND fecha = '2026-05-03';
```

Cassandra no debe entenderse como una versión distribuida de SQL Server. Es otro modelo de diseño.

Su fortaleza aparece cuando los datos son masivos, distribuidos, de alta escritura y con patrones de consulta conocidos. Su dificultad principal es que exige pensar antes qué se va a consultar. Una tabla mal diseñada puede obligar a lecturas costosas, uso indebido de filtros o duplicación desordenada de datos.

El aprendizaje central puede resumirse así:

En SQL se normaliza para representar correctamente el dominio.
En Cassandra se desnormaliza para responder eficientemente consultas específicas.

Y, especialmente para eventos masivos:

La clave de partición define cómo se agrupan y distribuyen los eventos.
La clave de clustering define cómo se ordenan dentro de cada grupo.
El tiempo suele formar parte del diseño, no solo del dato.

Una buena tabla Cassandra no es la que guarda todos los datos posibles, sino la que responde de manera eficiente una consulta importante.

<https://cassandra.apache.org/doc/4.1/index.html>

https://cassandra.apache.org/doc/4.1/cassandra/cql/cql_singlefile.html

Instalación con docker

Abrir **Docker Desktop** y esperá a que quede iniciado correctamente.

Cuando Docker Desktop esté abierto, debería aparecer como “running” o “Docker Desktop is running”. Recién después abris una terminal, puede ser **PowerShell**, **CMD** o la terminal de **VS Code**, y ejecutás los comandos.

```
docker pull cassandra:4.1.11
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\alfon> docker pull cassandra:4.1.11
4.1.11: Pulling from library/cassandra
c07c3e606820: Download complete
ff86ea2e5edc: Downloading [=>] 1.049MB/28.24MB
1e7ac21e111b: Download complete
49697762cb71: Pulling fs layer
69dc903843ff: Pulling fs layer
82283a83527a: Pulling fs layer
4d4316ae7ac9: Downloading [=>] 1.049MB/47.34MB
8c93b389327b: Download complete
e9165dffa48d4: Download complete
77620562c191: Downloading [=====]>] 1.049MB/3.282MB
```

Luego:

```
docker run --name cassandra-clase7 -d -p 9042:9042 cassandra:4.1.11
```

Y para entrar a Cassandra:

```
docker exec -it cassandra-clase7 cqlsh
```

```
PS C:\Users\alfon> docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS
ba609713c862   cassandra:4.1.11 "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp
PS C:\Users\alfon> docker exec -it cassandra-clase7 cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.11 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh>
```

Con eso ya se puede usar **cqlsh**, que es la consola para ejecutar comandos CQL. No hace falta instalar un cliente gráfico para la primera práctica.

Herramientas necesarias: Docker Desktop y la imagen oficial de Apache Cassandra 4.1.11.

Licencia: Apache Cassandra es software libre y open source bajo Apache License 2.0.

Modalidad de uso: se utilizará Cassandra mediante contenedor Docker para garantizar que todos trabajen con la misma versión.

Guía de laboratorio – Clase 7

Cassandra, CQL y diseño orientado a consultas

Objetivo del laboratorio

El objetivo de esta práctica es comprender cómo se trabaja con Cassandra desde la consola cqlsh, utilizando comandos básicos de CQL y diseñando tablas orientadas a consultas. La actividad parte de una tabla simple para entender la sintaxis y luego avanza hacia el diseño correcto para eventos masivos, aplicando claves de partición, claves de clustering y consultas eficientes.

La idea central de la práctica es la siguiente:

En Cassandra no se diseña primero una tabla general para consultar cualquier cosa. Primero se define qué consulta se necesita responder. Luego se crea una tabla optimizada para esa consulta.

El laboratorio toma como referencia la lógica de práctica con CREATE KEYSPACE, CREATE TABLE, INSERT, SELECT, consultas por clave primaria y diseño de tablas específicas para evitar ALLOW FILTERING, tal como aparece en la guía base de Cassandra .

1. Ingreso a Cassandra

Con el contenedor de Cassandra ya iniciado, ingresar a la consola de Cassandra con:

```
docker exec -it cassandra-clase7 cqlsh
```

Una vez dentro, se verá una consola similar a:

```
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.x.x | Cassandra 4.1.11 | CQL spec 3.x.x]
cqlsh>
```

Para verificar la conexión:

```
SHOW HOST;
```

Este comando permite confirmar que la consola está conectada al nodo Cassandra.

2. Crear el keyspace de trabajo

En Cassandra, un keyspace es el espacio lógico donde se agrupan las tablas de una aplicación o práctica. Se puede pensar como una unidad similar a una base de datos en otros motores, aunque Cassandra tiene su propia lógica distribuida.

Ejecutar:

```
CREATE KEYSPACE clase7_eventos
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
```

Luego seleccionar el keyspace:

```
USE clase7_eventos;
```

Para verificar que el keyspace fue creado:

```
DESCRIBE KEYSPACES;
```

También se puede consultar la estructura del keyspace actual con:

```
DESCRIBE KEYSPACE clase7_eventos;
```

Explicación

SimpleStrategy se utiliza en laboratorios locales con un solo nodo. replication_factor = 1 indica que cada dato tendrá una sola copia. Para un entorno real distribuido, este valor debería analizarse según disponibilidad, tolerancia a fallos y cantidad de nodos.

3. Crear una primera tabla simple

La primera tabla servirá para entender la sintaxis básica de CQL. Todavía no representa el mejor diseño Cassandra para eventos masivos.

Crear la tabla:

```
CREATE TABLE eventos (
    evento_id uuid PRIMARY KEY,
    usuario_id text,
    tipo_evento text,
    fecha date,
    evento_ts timestamp,
    resultado text,
    riesgo int
);
```

Verificar la estructura:

```
DESCRIBE TABLE eventos;
```

Explicación

La clave primaria es evento_id. Esto significa que Cassandra podrá recuperar eficientemente un evento si se conoce su identificador único.

En esta tabla, columnas como usuario_id, tipo_evento, fecha o riesgo son datos descriptivos, pero no forman parte de la clave primaria. Por lo tanto, no están pensadas como ruta principal de consulta.

4. Insertar datos de prueba

Insertar los siguientes eventos:

```
INSERT INTO eventos (evento_id, usuario_id, tipo_evento, fecha, evento_ts, resultado, riesgo)
```

```
VALUES (uuid(), 'USR1001', 'LOGIN', '2026-05-03', '2026-05-03 08:10:00', 'OK', 10);
```

```
INSERT INTO eventos (evento_id, usuario_id, tipo_evento, fecha, evento_ts, resultado, riesgo)
```

```
VALUES (uuid(), 'USR1001', 'LOGIN_FALLIDO', '2026-05-03', '2026-05-03 08:15:00', 'ERROR', 70);
```

```
INSERT INTO eventos (evento_id, usuario_id, tipo_evento, fecha, evento_ts, resultado, riesgo)
```

```
VALUES (uuid(), 'USR1001', 'CAMBIO_PASSWORD', '2026-05-03', '2026-05-03 08:25:00', 'OK', 40);
```

```
INSERT INTO eventos (evento_id, usuario_id, tipo_evento, fecha, evento_ts, resultado, riesgo)
```

```
VALUES (uuid(), 'USR1002', 'PAGO_RECHAZADO', '2026-05-03', '2026-05-03 09:00:00', 'ERROR', 85);
```

```
INSERT INTO eventos (evento_id, usuario_id, tipo_evento, fecha, evento_ts, resultado, riesgo)
```

```
VALUES (uuid(), 'USR1003', 'LOGIN_FALLIDO', '2026-05-03', '2026-05-03 09:10:00', 'ERROR', 65);
```

Consultar todos los registros:

```
SELECT * FROM eventos;
```

Observación importante

El comando uuid() genera automáticamente un identificador único. Cada vez que se ejecuta un INSERT, Cassandra crea un nuevo valor para evento_id.

5. Consultar por clave primaria

Como la tabla fue creada con evento_id como clave primaria, la consulta eficiente es por ese campo.

Primero ejecutar:

```
SELECT * FROM eventos;
```

Copiar uno de los valores de evento_id que aparezca en pantalla.

Luego ejecutar una consulta como la siguiente, reemplazando el valor por un UUID real obtenido en el paso anterior:

```
SELECT *
FROM eventos
WHERE evento_id = 00000000-0000-0000-0000-000000000000;
```

Explicación

Esta consulta es correcta porque usa la clave primaria de la tabla. Cassandra puede ubicar el dato directamente a partir de esa clave.

6. Probar una consulta problemática

Ahora intentar consultar por usuario:

```
SELECT *
FROM eventos
WHERE usuario_id = 'USR1001';
```

Es esperable que Cassandra rechace la consulta o indique que no puede ejecutarla sin filtrado. Esto ocurre porque usuario_id no forma parte de la clave primaria.

Análisis

En SQL Server sería normal consultar:

```
SELECT *
FROM eventos
WHERE usuario_id = 'USR1001';
```

Pero en Cassandra no alcanza con que la columna exista. La tabla debe estar diseñada para permitir esa consulta.

Esta diferencia es central: CQL se parece a SQL en la sintaxis, pero Cassandra no permite consultar libremente por cualquier columna si la tabla no fue diseñada para eso.

7. Uso de ALLOW FILTERING como advertencia

Ejecutar:

```
SELECT *
FROM eventos
```

```
WHERE usuario_id = 'USR1001'
ALLOW FILTERING;
```

La consulta puede devolver resultados, pero no representa una buena práctica para eventos masivos.

Explicación

ALLOW FILTERING le indica a Cassandra que permita una búsqueda más costosa. Puede servir para una prueba pequeña, pero no debe ser la solución habitual. En Cassandra, la respuesta correcta no es forzar la consulta, sino diseñar una tabla adecuada.

La guía de referencia usada por otros docentes también muestra esta idea: primero aparece una consulta con ALLOW FILTERING y luego se propone crear una tabla específica para consultar por el campo necesario, evitando ese filtrado .

8. Crear una tabla orientada a consulta

Ahora se diseñará una tabla correcta para responder esta consulta:

Consultar los eventos de un usuario en una fecha determinada, ordenados desde el más reciente al más antiguo.

Crear la tabla:

```
CREATE TABLE eventos_por_usuario_dia (
  usuario_id text,
  fecha date,
  evento_ts timestamp,
  evento_id uuid,
  tipo_evento text,
  resultado text,
  riesgo int,
  PRIMARY KEY ((usuario_id, fecha), evento_ts, evento_id)
) WITH CLUSTERING ORDER BY (evento_ts DESC);
```

Explicación del diseño

La clave primaria es:

```
PRIMARY KEY ((usuario_id, fecha), evento_ts, evento_id)
```

Esto significa:

Clave de partición: usuario_id + fecha

Clave de clustering: evento_ts + evento_id

La clave de partición agrupa los eventos de un mismo usuario en un mismo día.
 La clave de clustering ordena los eventos dentro de esa partición.
 El orden descendente permite consultar primero los eventos más recientes.

9. Insertar datos en la tabla orientada a consulta

Insertar los siguientes eventos:

```
INSERT INTO eventos_por_usuario_dia (usuario_id, fecha, evento_ts, evento_id,
tipo_evento, resultado, riesgo)
VALUES ('USR1001', '2026-05-03', '2026-05-03 08:10:00', uuid(), 'LOGIN', 'OK', 10);
```

```
INSERT INTO eventos_por_usuario_dia (usuario_id, fecha, evento_ts, evento_id,
tipo_evento, resultado, riesgo)
VALUES ('USR1001', '2026-05-03', '2026-05-03 08:15:00', uuid(), 'LOGIN_FALLIDO',
'ERROR', 70);
```

```
INSERT INTO eventos_por_usuario_dia (usuario_id, fecha, evento_ts, evento_id,
tipo_evento, resultado, riesgo)
VALUES ('USR1001', '2026-05-03', '2026-05-03 08:25:00', uuid(),
'CAMBIO_PASSWORD', 'OK', 40);
```

```
INSERT INTO eventos_por_usuario_dia (usuario_id, fecha, evento_ts, evento_id,
tipo_evento, resultado, riesgo)
VALUES ('USR1002', '2026-05-03', '2026-05-03 09:00:00', uuid(), 'PAGO_RECHAZADO',
'ERROR', 85);
```

```
INSERT INTO eventos_por_usuario_dia (usuario_id, fecha, evento_ts, evento_id,
tipo_evento, resultado, riesgo)
VALUES ('USR1003', '2026-05-03', '2026-05-03 09:10:00', uuid(), 'LOGIN_FALLIDO',
'ERROR', 65);
```

10. Consultar eventos por usuario y día

Ejecutar:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03';
```

El resultado debería mostrar los eventos del usuario USR1001 para esa fecha, ordenados desde el más reciente al más antiguo.

Luego ejecutar:


```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03'
LIMIT 2;
```

Explicación

LIMIT 2 restringe la cantidad de filas devueltas. En este caso, como la tabla está ordenada por evento_ts DESC, se obtienen los dos eventos más recientes de ese usuario en ese día.

11. Probar una consulta incompleta

Ejecutar:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001';
```

Esta consulta no debería ser válida porque no se está usando la clave de partición completa. La clave de partición está compuesta por:

usuario_id + fecha

Por eso Cassandra espera recibir ambos valores.

Conclusión del paso

Cuando una clave de partición está compuesta por más de una columna, la consulta debe incluir todas esas columnas para acceder eficientemente a la partición.

12. Crear otra tabla para otra consulta

Ahora se necesita responder otra pregunta:

Consultar eventos por tipo de evento y fecha.

Por ejemplo:

¿Qué eventos LOGIN_FALLIDO ocurrieron el 2026-05-03?

Crear otra tabla:

```
CREATE TABLE eventos_por_tipo_fecha (
    tipo_evento text,
    fecha date,
    evento_ts timestamp,
```

```
evento_id uuid,  
usuario_id text,  
resultado text,  
riesgo int,  
PRIMARY KEY ((tipo_evento, fecha), evento_ts, evento_id)  
) WITH CLUSTERING ORDER BY (evento_ts DESC);
```

Explicación del diseño

La clave de partición ahora es:

tipo_evento + fecha

Esto permite agrupar los eventos por tipo y día.

La clave de clustering sigue usando evento_ts, para ordenar los eventos por tiempo.

13. Insertar datos en la segunda tabla orientada a consulta

Insertar los mismos eventos relevantes, pero ahora organizados para otra consulta:

```
INSERT INTO eventos_por_tipo_fecha (tipo_evento, fecha, evento_ts, evento_id,  
usuario_id, resultado, riesgo)  
VALUES ('LOGIN', '2026-05-03', '2026-05-03 08:10:00', uuid(), 'USR1001', 'OK', 10);
```

```
INSERT INTO eventos_por_tipo_fecha (tipo_evento, fecha, evento_ts, evento_id,  
usuario_id, resultado, riesgo)  
VALUES ('LOGIN_FALLIDO', '2026-05-03', '2026-05-03 08:15:00', uuid(), 'USR1001',  
'ERROR', 70);
```

```
INSERT INTO eventos_por_tipo_fecha (tipo_evento, fecha, evento_ts, evento_id,  
usuario_id, resultado, riesgo)  
VALUES ('CAMBIO_PASSWORD', '2026-05-03', '2026-05-03 08:25:00', uuid(),  
'USR1001', 'OK', 40);
```

```
INSERT INTO eventos_por_tipo_fecha (tipo_evento, fecha, evento_ts, evento_id,  
usuario_id, resultado, riesgo)  
VALUES ('PAGO_RECHAZADO', '2026-05-03', '2026-05-03 09:00:00', uuid(), 'USR1002',  
'ERROR', 85);
```

```
INSERT INTO eventos_por_tipo_fecha (tipo_evento, fecha, evento_ts, evento_id,  
usuario_id, resultado, riesgo)  
VALUES ('LOGIN_FALLIDO', '2026-05-03', '2026-05-03 09:10:00', uuid(), 'USR1003',  
'ERROR', 65);
```

14. Consultar eventos por tipo y fecha

Ejecutar:

```
SELECT *
FROM eventos_por_tipo_fecha
WHERE tipo_evento = 'LOGIN_FALLIDO'
AND fecha = '2026-05-03';
```

Luego limitar la cantidad de resultados:

```
SELECT *
FROM eventos_por_tipo_fecha
WHERE tipo_evento = 'LOGIN_FALLIDO'
AND fecha = '2026-05-03'
LIMIT 1;
```

Análisis

La misma información fue cargada en más de una tabla. Esto no es un error. En Cassandra es habitual desnormalizar los datos para responder distintas consultas de forma eficiente.

Una tabla responde: Eventos por usuario y día.

Otra tabla responde: Eventos por tipo y fecha.

15. Actualizar un dato

Actualizar el nivel de riesgo de un evento requiere conocer su clave primaria completa. Primero consultar los eventos de USR1001:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03';
```

Copiar el valor de evento_ts y evento_id de uno de los registros.

Luego ejecutar un UPDATE reemplazando los valores correspondientes:

```
UPDATE eventos_por_usuario_dia
SET riesgo = 90
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03'
AND evento_ts = '2026-05-03 08:15:00'
AND evento_id = 00000000-0000-0000-0000-000000000000;
```

Verificar el cambio:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03';
```

Importante

En Cassandra, para actualizar una fila concreta, se debe identificar correctamente la fila mediante su clave.

16. Eliminar un registro

Para eliminar un registro también se necesita su clave.

Ejemplo:

```
DELETE FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03'
AND evento_ts = '2026-05-03 08:15:00'
AND evento_id = 00000000-0000-0000-0000-000000000000;
```

Después consultar nuevamente:

```
SELECT *
FROM eventos_por_usuario_dia
WHERE usuario_id = 'USR1001'
AND fecha = '2026-05-03';
```

Observación

En Cassandra, las eliminaciones tienen una lógica interna particular, porque los datos se marcan inicialmente como eliminados mediante tombstones y se eliminan físicamente más adelante durante procesos internos de compactación. Para esta práctica alcanza con comprender el efecto visible del DELETE.

17. Vaciar una tabla

Para borrar todos los registros de una tabla sin eliminar la estructura:

```
TRUNCATE eventos;
```

Luego verificar:

```
SELECT * FROM eventos;
```

TRUNCATE borra los datos, pero mantiene la tabla creada.

18. Eliminar tablas y keyspace

Al finalizar la práctica, si se desea limpiar el entorno:

```
DROP TABLE IF EXISTS eventos;
DROP TABLE IF EXISTS eventos_por_usuario_dia;
DROP TABLE IF EXISTS eventos_por_tipo_fecha;
```

Luego salir del keyspace o directamente eliminarlo:

```
DROP KEYSPACE IF EXISTS clase7_eventos;
```

Actividad de cierre

Como vimos en esta práctica, Cassandra no se utiliza como una base relacional tradicional. Aunque CQL usa una sintaxis parecida a SQL, el diseño depende de las consultas que se deben responder.

El aprendizaje principal es:

Consulta esperada → clave de partición → clave de clustering → tabla optimizada

Una buena tabla Cassandra no es necesariamente la más normalizada ni la más general. Es aquella que permite recuperar los datos requeridos de forma eficiente, con una clave bien diseñada y evitando búsquedas innecesarias.

Responder brevemente:

1. ¿Por qué la tabla eventos no es suficiente para consultar eficientemente por usuario?
2. ¿Qué diferencia hay entre evento_id como clave primaria y (usuario_id, fecha) como clave de partición?
3. ¿Por qué se creó una tabla para consultar por usuario y otra para consultar por tipo de evento?
4. ¿Qué función cumple evento_ts como clave de clustering?
5. ¿Por qué ALLOW FILTERING no debería ser la solución principal?
6. ¿Qué ventaja tiene diseñar tablas a partir de consultas conocidas?